

Lab Assignment-10.1

2303A54012
A.Sanjay B47A

Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability Lab Objectives

- Use AI for automated code review and quality enhancement.
- Identify and fix syntax, logical, performance, and security issues in Python code.
- Improve readability and maintainability through structured refactoring and comments.
- Apply prompt engineering for targeted improvements.
- Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student def
calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)    return
avrage # Typo here marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Prompt:-

Review the following Python code and fix all syntax and logical errors. Explain what was wrong and provide corrected runnable code.

Code(python):-

```
1 # Calculate average score of a student
2 def calc_average(marks):
3     total = 0
4     for m in marks:
5         total += m
6     average = total / len(marks)
7     return average # Fixed typo
8 marks = [85, 90, 78, 92]
9 print("Average Score is", calc_average(marks))
10
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
Average Score is 86.25
```

```
>>>
```

Explanation of Fixes:

1. Fixed indentation of average and return.
2. Corrected typo: avrage → average.
3. Added missing closing parenthesis in print().
4. Ensured consistent spacing and formatting.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20)) Expected
```

Output:

- Well-formatted PEP 8-compliant Python code.

Prompt:-

Refactor the following Python code to follow PEP 8 style guidelines. Improve formatting, spacing, and naming conventions.

Code(python):-

```
1 def area_of_rectangle(length, breadth):
2     """Return the area of a rectangle."""
3     return length * breadth
4
5 print(area_of_rectangle(10, 20))
6
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
200
>>>
```

Improvements Made:

- Used descriptive variable names.
- Added docstring.
- Proper spacing.
- Removed one-line function definition.
- Followed PEP 8 naming conventions.

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code: def

c(x,y):

return x*y/100

a=200 b=15

print(c(a,b))

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

Prompt:-

Improve the readability of the following code without changing its logic. Use meaningful variable names and add comments.

Code(python):-

```
1 def calculate_percentage_amount(total_amount, percentage):
2     """ Calculate percentage value of a given amount. """
3     return total_amount * percentage / 100
4
5 principal_amount = 200
6 interest_rate = 15
7 result = calculate_percentage_amount(principal_amount, interest_rate)
8 print("Calculated Value:", result)
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
Calculated Value: 30.0
```

```
>>>
```

Improvements:

- Function renamed to meaningful name.
- Variables renamed clearly.
- Added comments and spacing.
- Improved output clarity.

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]  
print("Welcome", students[0])  
print("Welcome", students[1])  
print("Welcome", students[2]) Expected
```

Output:

- Modular code with reusable functions.

Prompt:-

Refactor the repetitive code into reusable functions to improve maintainability.

Code(python):-

```
1 def welcome_student(name):  
2     """Print welcome message for a student."""  
3     print("Welcome", name)  
4  
5 students = ["Alice", "Bob", "Charlie"]  
6 for student in students:  
7     welcome_student(student)  
8
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
Welcome Alice
```

```
Welcome Bob
```

```
Welcome Charlie
```

```
>>>
```

Improvements:

- Removed repetitive print statements.
- Created reusable function.
- Used loop for scalability.
- Code works for any number of students.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code: # Find squares

of numbers `nums = [i for i in`

`range(1,1000000)] squares = []` for

`n in nums:`

`squares.append(n**2)`

`print(len(squares))` Expected

Output:

- Optimized code using list comprehensions or vectorized operations.

Prompt:-

Optimize the following Python code to improve performance using list comprehension.

Code(python):-

```
1 # Find squares of numbers
2 squares = [n ** 2 for n in range(1, 1_000_000)]
3
4 print(len(squares))
5
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
999999
```

```
>>>
```

Improvements:

- Removed unnecessary `nums` list.
- Used direct list comprehension.
- Reduced memory usage.

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):  
    if  
    score >= 90:        return  
    "A"    else:        if score  
    >= 80:        return "B"  
    else:        if score >=  
    70:        return  
    "C"        else:  
    if score >= 60:  
    return "D"  
    else:  
        return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Prompt:-

Simplify the following grading function using cleaner logic such as elif statements.

Code(python):-

```

1 def grade(score):
2     """ Return the grade based on the given score."""
3     if score >= 90:
4         return "A"
5     elif score >= 80:
6         return "B"
7     elif score >= 70:
8         return "C"
9     elif score >= 60:
10        return "D"
11    else:
12        return "F"
13
14 # Example usage
15 student_score = 85
16 result = grade(student_score)
17 print("Score:", student_score)
18 print("Grade:", result)
19

```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
Score: 85
```

```
Grade: B
```

```
>>>
```

Improvements:

- Replaced nested if-else with elif.
- Fixed syntax error ("F"" → "F").
- Improved readability.
- Reduced indentation complexity